

IP.LSH.DBSCAN: Integrated Parallel Density-Based Clustering through Locality-Sensitive Hashing

Amir Keramatian , Vincenzo Gulisano , Marina Papatriantafilou, and
Philippas Tsigas

Chalmers University of Technology, Gothenburg, Sweden
{amirke,vincenzo.gulisano,ptrianta,tsigas}@chalmers.se

The artifact is a prototype for approximate parallel DBSCAN using locality sensitive hashing.

Software Requirements

We bench-marked the artifact on a fresh installation of Ubuntu 20.04 on Amazon Web Services using a c5.18xlarge machine. The software dependencies are GSL (GNU Scientific Library), TBB (Threading Building Blocks), POSIX pthreads, and GOMP. Below shows the installed version of the aforementioned libraries on the Amazon machine which we used for bench-marking.

```
ubuntu@ip-172-31-4-142:~$ apt list --installed | grep 'gsl\|pthread\|gomp\|tbb\|pthread'
WARNING: apt does not have a stable CLI interface. Use with caution in scripts.

libevent-pthreads-2.1-7/focal,now 2.1.11-stable-1 amd64 [installed,automatic]
libgomp1/focal-updates,focal-security,now 10.3.0-1ubuntu1~20.04 amd64 [installed,automatic]
libgsl-dev/focal,now 2.5+dfsg-6build1 amd64 [installed]
libgsl23/focal,now 2.5+dfsg-6build1 amd64 [installed,automatic]
libgslcblas0/focal,now 2.5+dfsg-6build1 amd64 [installed,automatic]
libtbb-dev/focal,now 2020.1-2 amd64 [installed]
libtbb2/focal,now 2020.1-2 amd64 [installed,automatic]
```

In addition to the aforementioned libraries, we utilized a third-party software to compute the *rand index accuracy* of our method in comparison with exact DBSCAN algorithms (e.g., references [13,22,31] in the manuscript). The source of the third party software that we use to calculate the rand index values can be found in the following git repository:

<https://github.com/bjoern-andres/partition-comparison.git>

For our convenience, we adapted the aforementioned software to create a command line tool as the following:

```
./computeAccuracy_fast [index_file_1] [index_file_2] [output_file]
```

Where `index_file_1` and `index_file_2` are two files containing line-separated clustering indexes created by approximate and exact algorithms (the order is not important). The corresponding rand index value will be added as a new line at the end of `output_file`.

We use the aforementioned command line tool to calculate the rand index accuracy of our experiments. We attach the adapted project for your convenience, but you are more than welcome to use the original software given in the aforementioned git repository or any other software that can calculate rand index accuracy.

How to Build

Navigate to project's root folder (where CmakeLists.txt exists), and issue the following commands:

mkdir build; cd build; cmake ..; make -j

```
ubuntu@ip-172-31-4-142:~/artifact/IP.LSH.DBSCAN-main$ mkdir build; cd build; cmake ..; make -j
-- The C compiler identification is GNU 9.3.0
-- The CXX compiler identification is GNU 9.3.0
-- Check for working C compiler: /usr/bin/cc
-- Check for working C compiler: /usr/bin/cc -- works
-- Detecting C compiler ABI info
-- Detecting C compiler ABI info - done
-- Detecting C compile features
-- Detecting C compile features - done
-- Check for working CXX compiler: /usr/bin/c++
-- Check for working CXX compiler: /usr/bin/c++ -- works
-- Detecting CXX compiler ABI info
-- Detecting CXX compiler ABI info - done
-- Detecting CXX compile features
-- Detecting CXX compile features - done
-- *****Build type: Release*****
-- Configuring done
-- Generating done
-- Build files have been written to: /home/ubuntu/artifact/IP.LSH.DBSCAN-main/build
Scanning dependencies of target myLib
[ 5%] Building CXX object CMakeFiles/myLib.dir/src/ConcurrentLSHDBSCAN.cc.o
[ 11%] Building CXX object CMakeFiles/myLib.dir/src/VanillaDBSCAN_LSH.cc.o
[ 23%] Building CXX object CMakeFiles/myLib.dir/src/LSHDBSCAN.cc.o
[ 23%] Building CXX object CMakeFiles/myLib.dir/src/VanillaDBSCAN.cc.o
[ 35%] Building CXX object CMakeFiles/myLib.dir/src/globals.cc.o
[ 35%] Building CXX object CMakeFiles/myLib.dir/src/point.cc.o
[ 41%] Building CXX object CMakeFiles/myLib.dir/src/randomGen.cc.o
[ 47%] Building CXX object CMakeFiles/myLib.dir/src/transformation.cc.o
[ 52%] Linking CXX static library libmyLib.a
[ 52%] Built target myLib
Scanning dependencies of target ParameterExploration_exec
Scanning dependencies of target LSHDBSCAN_exec
Scanning dependencies of target Vanila_DBSCAN_exec
Scanning dependencies of target Vanila_DBSCAN_LSH_exec
[ 58%] Building CXX object CMakeFiles/ParameterExploration_exec.dir/apps/main_parameter_exploration.cc.o
[ 64%] Building CXX object CMakeFiles/Vanila_DBSCAN_exec.dir/apps/main_vanilla_dbscan.cc.o
[ 70%] Building CXX object CMakeFiles/LSHDBSCAN_exec.dir/apps/main_general.cc.o
[ 76%] Building CXX object CMakeFiles/Vanila_DBSCAN_LSH_exec.dir/apps/main_vanilla_dbscan_lsh.cc.o
[ 82%] Linking CXX executable Vanila_DBSCAN_exec
[ 82%] Built target Vanila_DBSCAN_exec
/home/ubuntu/artifact/IP.LSH.DBSCAN-main/apps/main_general.cc: In function 'int main(int, char*)':
/home/ubuntu/artifact/IP.LSH.DBSCAN-main/apps/main_general.cc:65:13: warning: ignoring return value of 'int system(const char*)', declared with attribute warn_unused_result [-Wunused-result]
   65 |         system(accuracyCommand.c_str());
      |         ^
/home/ubuntu/artifact/IP.LSH.DBSCAN-main/apps/main_parameter_exploration.cc: In function 'int main(int, char*)':
/home/ubuntu/artifact/IP.LSH.DBSCAN-main/apps/main_parameter_exploration.cc:78:14: warning: ignoring return value of 'int system(const char*)', declared with attribute warn_unused_result [-Wunused-result]
   78 |         system(accuracyCommand.c_str());
      |         ^
/home/ubuntu/artifact/IP.LSH.DBSCAN-main/apps/main_vanilla_dbscan_lsh.cc: In function 'int main(int, char*)':
/home/ubuntu/artifact/IP.LSH.DBSCAN-main/apps/main_vanilla_dbscan_lsh.cc:51:13: warning: ignoring return value of 'int system(const char*)', declared with attribute warn_unused_result [-Wunused-result]
   51 |         system(accuracyCommand.c_str());
      |         ^
[ 88%] Linking CXX executable Vanila_DBSCAN_LSH_exec
[ 94%] Linking CXX executable LSHDBSCAN_exec
[ 94%] Built target Vanila_DBSCAN_LSH_exec
[ 94%] Built target LSHDBSCAN_exec
[100%] Linking CXX executable ParameterExploration_exec
[100%] Built target ParameterExploration_exec
ubuntu@ip-172-31-4-142:~/artifact/IP.LSH.DBSCAN-main/build$
```

A successful build will lead to creation of 4 executable binaries:

1. *Vanila_DBSCAN_exec*: This is our implementation of plain exact sequential DBSCAN algorithm.
2. *Vanila_DBSCAN_LSH_exec*: This is our implementation of approximate sequential DBSCAN algorithm proposed in references [25,34], where the idea is to use LSH for epsilon queries (see the manuscript).
3. *ParameterExploration_exec*: This binary executes IP.LSH.DBSCAN with varying parameters and logs the accuracies.
4. *LSHDBSCAN_exec*: This binary is the **IP.LSH.DBSCAN** algorithm. It can be executed as a command line.

Usage

LSHDBSCAN_exec can take several parameters:

1. **-f** *inputFile*: *inputFile* is the full address and the file name of the dataset to cluster. Each row in the dataset must correspond to one observation (one point). Different features (i.e., dimensions) for each observation must be separated by space.
2. **-L** *#HashTables*: We can set the number of LSH tables with this parameter (check the manuscript).
3. **-M** *#HashFunctionsPerTable*: We can set the number of hash functions per table with this parameter (check the manuscript).
4. **-t** *#threads*: We can set the number of threads with this parameter.
5. **-e** *eps*: We can set the epsilon value (DBSCAN's parameter) with this parameter.
6. **-m** *minPts*: We can set the minPts value (DBSCAN's parameter) with this parameter.
7. **-b** *baselineClusteringFile*: We can choose the baseline (exact) clustering index file with this option to calculate the clustering accuracy (which will be concatenated at the end of a file named *accuracy.txt*).
8. **-a**: This option does not require a value. If set, the algorithm utilizes the angular distance; otherwise, the algorithm uses Euclidean distance by default.

Each time LSHDBSCAN_exec finishes clustering, it appends its execution log as a line at the end of a file named *benchmark.txt*. Execution log consists of duration of different phases of the algorithm. Therefore, one way to calculate how long each execution takes is to sum up the corresponding entries added in the *benchmark.txt* file.

Bench-marking IP.LSH.DBSCAN

Benchmarked datasets (and their clustering indexes using exact DBSCAN with manuscript's defaults parameters) can be found in the following link:

https://drive.google.com/drive/mobile/folders/12pflqbkR8PuhvW_9dkuRqa2kQ0GSV-KE?usp=%5C=sharing

Navigate to the *benchmarks* folder. Firstly, make sure the copy of computeAccuracy_fast binary (see §Software Requirements) is executable (or build the binary yourself). In this folder, **script_LSHDBSCAN_general.sh** is the key to replicating the manuscript's results. For each dataset, there is a line. Simply uncomment the desired line and make sure the lines corresponding to other datasets are commented. For example, the configuration in the attached screenshot will run IP.LSH.DBSCAN on the household dataset with epsilon=2000, minPts=100, L=20, M=5, which corresponds to the yellow plot in Fig. 3 (b) in the manuscript. After bench-marking with the aforementioned parameters are completed, the script will rename the files. Repeat the process with {L=10, M=5}, and {L=5, M=5} (keep epsilon and minPts unchanged) to gather the results corresponding to the dark blue and orange plots in Fig. 2 (b). The remainder of the results in Fig. 2(b) correspond to the other bench-marked methods (competitors) which can be obtained as open source software. Once all the data is gathered, use the attached Matlab script (completionTimes_EuroPar.m) to plot the results. Make sure the measurements are organized into folders with the following hierarchy:

```
amazon
    household
        LSHDBSCAN
        TEDBSCAN
        HPDBSCAN
        PDSDBSCAN
```

```
#!/bin/bash

#epsilon=1500;minPts=100;numberOfHashTables=10;numberOfHyperPlanesPerTable=5;files=" -f /data/cleaned_day_0.txt"

epsilon=2000;minPts=100;numberOfHashTables=20;numberOfHyperPlanesPerTable=5;files=" -f ../../datasets/household/clean_normalized_household.txt -b ../../datasets/household/clean_normalized_household.idx"

#epsilon=0.001;minPts=500;numberOfHashTables=20;numberOfHyperPlanesPerTable=2;files=" -f ../../datasets/geolife/geolife_small.txt -b ../../datasets/geolife/geolife_small.idx"

for numberOfThreads in 1 2 3 4 5 10 15 20 25 30 35 40 45 50 55 60 65 70
do
    params=" -m ${minPts} -e ${epsilon} -t ${numberOfThreads} -M ${numberOfHyperPlanesPerTable} -L ${numberOfHashTables}"
    command="../../build/LSHDBSCAN_exec"$files$params
    $command
done

mv accuracy.txt ./accuracy_L_${numberOfHashTables}_M_${numberOfHyperPlanesPerTable}.txt
mv benchmark.txt ./benchmark_L_${numberOfHashTables}_M_${numberOfHyperPlanesPerTable}.txt
mv memory_usage.log ./memory_usage_L_${numberOfHashTables}_M_${numberOfHyperPlanesPerTable}.log
```

Note. For convenience, three dedicated script files are provided in the *benchmarks* folder to replicate the IP.LSH.DBSCAN results in Fig. 3 (a), Fig. 3 (b), and Fig. 3 (c) of the manuscript.

Bench-marking Competitors

TEDBSCAN

TEDBSCAN abbreviates Theoretically-Efficient and Practical Parallel DBSCAN [1].

Link to source code: <https://github.com/wangyiqu/dbscan-python>

Commit version: [be4871c](#)

Important Notes: Make sure the format of the input dataset to this algorithm is compatible with the requirements of its implementation. For example, the implementation of this algorithm silently discards data points containing non-decimal values (for example when the values are represented in scientific format). Furthermore, the following line must appear on the first line of the input dataset: *dim [number of dimensions of the input dataset]*

HPDBSCAN

HPDBSCAN abbreviates Highly parallel DBSCAN [2].

Link to source code: <https://github.com/Markus-Goetz/hpdbname>

Commit version: [ae0c74c](#)

Notes: The implementation of this algorithm requires the data to be presented in h5 format.

PDSDBSCAN

PDSDBSCAN refers to a new scalable parallel DBSCAN algorithm using the disjoint-set data structure [3].

Link to source code: <http://users.eecs.northwestern.edu/~mpatwary/Software.html>

References

1. Y. Wang, Y. Gu, and J. Shun. Theoretically-efficient and practical parallel DBSCAN. In 2020 SIGMOD Int. Conf. on Management of Data, pages 2555–2571. ACM, 2020.
2. M. Götz, C. Bodenstein, and M. Riedel. HPDBSCAN: highly parallel DBSCAN. In Workshop on Machine Learning in High-Perf. Comp. Environments, MLHPC 2015, pages 2:1–2:10. ACM, 2015.
3. M. M. A. Patwary, D. Palsetia, A. Agrawal, W. Liao, F. Manne, and A. N. Choudhary. A new scalable parallel DBSCAN algorithm using the disjoint-set data structure. In SC Conf. on High Perf. Comp. Networking, Storage and Analysis, SC '12, page 62. IEEE/ACM, 2012.